

DeepBlue Rescue

1. Descripción breve

DeepBlue Rescue es la capa de persistencia de una plataforma para organizaciones dedicadas al rescate y rehabilitación de fauna marina. El proyecto modela el flujo completo de un caso de rescate: desde que un centro registra el hallazgo de un animal herido, pasando por su expediente médico y los tratamientos recibidos, hasta la participación de los especialistas involucrados en su recuperación.

Este laboratorio está dedicado exclusivamente a la persistencia: entidades JPA, migraciones Flyway, repositories con Spring Data JPA y pruebas de integración contra PostgreSQL real mediante Testcontainers. No incluye Controller, REST API, Service, DTO, Spring Security ni Frontend.

Stack: Java 21 · Spring Boot 4 · Spring Data JPA · Hibernate · Flyway · PostgreSQL · Testcontainers · Maven

2. Modelo de datos

Entidades

Entidad	Tabla	Descripción
RescueCenter	rescue_centers	Centro de recuperación de fauna marina
RescueCase	rescue_cases	Caso de rescate de un animal
Animal	animals	Animal rescatado
MedicalRecord	medical_records	Expediente médico del animal
Specialist	specialists	Especialista que interviene en la recuperación
Expertise	expertise	Área de experiencia de un especialista
Treatment	treatments	Tratamiento realizado a un animal

```
RESCUE_CENTER {  
    bigint id PK  
    varchar code UK  
    varchar name  
    varchar city  
}
```

```
RESCUE_CASE {  
    bigint id PK  
    varchar case_code UK  
    date rescue_date  
    varchar rescue_location  
    varchar status  
    bigint rescue_center_id FK  
}
```

```
ANIMAL {  
    bigint id PK  
    varchar animal_code UK  
    varchar common_name  
    varchar scientific_name  
    varchar sex  
    varchar tracking_device_code UK  
    bigint rescue_case_id FK UK  
}
```

```
MEDICAL_RECORD {  
    bigint id PK  
    bigint animal_id FK UK  
    decimal initial_weight  
    varchar initial_condition  
    text injuries  
    text observations  
}
```

```
SPECIALIST {  
    bigint id PK  
    varchar professional_code UK  
    varchar first_name  
    varchar last_name  
    varchar email UK  
    boolean active  
}
```

```
EXPERTISE {  
    bigint id PK  
    varchar name UK  
}
```

```
TREATMENT {  
    bigint id PK  
    bigint animal_id FK
```

```

    bigint specialist_id FK
    timestamp performed_at
    varchar type
    text description
}

```

Tablas de entidades vs. tablas asociativas

- **Tablas de entidades:** rescue_centers, rescue_cases, animals, medical_records, specialists, expertise, treatments.
- **Tabla asociativa:** specialist_expertise (resuelve la relación N:M entre Specialist y Expertise).

3. Relaciones

Relación	Cardinalidad	Dueño de la FK	Anotaciones clave
RescueCenter → RescueCase	1:N	RescueCase (rescue_center_id)	@OneToMany(mappedBy) / @ManyToOne + @JoinColumn
RescueCase → Animal	1:1	Animal (rescue_case_id, con UNIQUE)	@OneToOne(mappedBy) en RescueCase / @OneToOne + @JoinColumn(unique = true) en Animal
Animal → MedicalRecord	1:1	MedicalRecord (animal_id, con UNIQUE)	@OneToOne(mappedBy, cascade = ALL, orphanRemoval = true) en Animal
Specialist ↔ Expertise	N:M	Tabla intermedia specialist_expertise	@ManyToMany + @JoinTable en Specialist / @ManyToMany(mappedBy) en Expertise

Animal → Treatment	1:N	Treatment (animal_id)	@OneToMany(mappedBy) en Animal / @ManyToOne en Treatment
Specialist → Treatment	1:N	Treatment (specialist_id)	@ManyToOne en Treatment

Notas de diseño:

- El UNIQUE sobre rescue_case_id en animals y sobre animal_id en medical_records es lo que convierte una FK ordinaria en una verdadera relación 1:1 a nivel de base de datos — sin esa constraint, PostgreSQL permitiría múltiples filas hijas apuntando al mismo padre (una 1:N disfrazada).
- Specialist y Expertise necesitan tabla intermedia porque ninguna de las dos tablas puede alojar una FK de cardinalidad múltiple hacia la otra: un especialista tiene varias áreas de experiencia, y una misma área es compartida por varios especialistas.
- Un Treatment no puede existir sin Animal ni sin Specialist — ambas FKs son NOT NULL, reflejando que todo tratamiento pertenece a un animal y fue realizado por alguien.
- tracking_device_code es NULL-able pero UNIQUE cuando existe (agregado en V3, ver sección de Flyway) — en PostgreSQL, múltiples valores NULL no violan una constraint UNIQUE.

4. Instrucciones para ejecutar

Requisitos previos

- Java 21
- Maven
- Docker (Docker Desktop en Windows/Mac, o Docker Engine en Linux) — **debe estar corriendo**, ya que Testcontainers lo necesita tanto para los tests como, opcionalmente, para levantar PostgreSQL en desarrollo local.

Configuración

El application.yml toma la configuración de la base de datos desde variables de entorno, con valores por defecto para desarrollo local:

datasource:

url: \${DB_URL:jdbc:postgresql://localhost:5432/deepblue}

username: \${DB_USER:postgres}

password: \${DB_PASSWORD:postgres}

Levanta un PostgreSQL local (por ejemplo, con `docker run -e POSTGRES_PASSWORD=postgres -p 5432:5432 -d postgres:18-alpine`) o ajusta las variables de entorno a tu propia instancia.

Compilar y ejecutar

`mvn clean install`

`mvn spring-boot:run`

5. Instrucciones para ejecutar tests

`mvn clean test`

Los tests usan **Testcontainers**, por lo que Docker debe estar activo antes de correr el comando. Puedes verificarlo con:

`docker ps`

Durante la ejecución de `PersistenceIntegrationTest`, verás temporalmente un contenedor postgres creado automáticamente por Testcontainers — se destruye al finalizar la suite.

El resultado esperado es:

BUILD SUCCESS

6. Flyway

Flyway es responsable de crear y evolucionar el esquema de base de datos mediante migraciones versionadas y ordenadas, en lugar de dejar que Hibernate lo genere automáticamente (`ddl-auto: create` o `update`).

En este proyecto se usa:

jpa:

hibernate:

ddl-auto: validate

validate le indica a Hibernate que **compare** el esquema mapeado por las entidades JPA contra el esquema real de la base de datos, y falle si no coinciden — pero nunca lo modifica. Quien crea y modifica las tablas es exclusivamente Flyway, a través de sus migraciones:

Migración	Contenido
V1__create_schema.sql	Creación de las 8 tablas, PKs, FKs, UNIQUE y CHECK constraints
V2__insert_expertise_catalog.sql	Catálogo semilla de áreas de experiencia (Trauma, Rehabilitation, Marine Reptiles, Marine Mammals, Marine Birds, etc.)
V3__add_tracking_device_to_animal.sql	Agrega tracking_device_code (nullable, UNIQUE) a animals, sin modificar V1 ya aplicada

Este enfoque garantiza que el esquema evoluciona de forma controlada y reproducible en cualquier entorno (desarrollo, pruebas, producción), y que nunca se pierde el historial de cambios de la base de datos.

7. Testcontainers

Testcontainers permite ejecutar las pruebas de integración contra una instancia **real** de PostgreSQL, levantada automáticamente en un contenedor Docker, en lugar de usar una base de datos en memoria como H2 (prohibido por las reglas del laboratorio).

Esto importa porque una base de datos en memoria no aplica las mismas reglas, tipos de datos ni comportamiento de constraints que PostgreSQL real — por ejemplo, cómo maneja NULL frente a UNIQUE, o el comportamiento exacto de un CHECK. Probar contra el motor real evita falsos positivos.

En el proyecto:

@Testcontainers

@SpringBootTest

@Transactional

class PersistenceIntegrationTest {

@Container

@ServiceConnection

static final PostgreSQLContainer<?> postgres =

new PostgreSQLContainer<>(DockerImageName.parse("postgres:18-alpine"))

.withDatabaseName("deepblue_test")

.withUsername("deepblue")

.withPassword("deepblue");

// ...

}

@ServiceConnection conecta automáticamente el DataSource de Spring Boot al contenedor, sin necesidad de configurar manualmente URL, usuario o contraseña por separado. Al ejecutar mvn clean test, Flyway corre sus migraciones contra este PostgreSQL real antes de que Hibernate valide el esquema y las pruebas se ejecuten.

8. Query Methods implementados

Repository	Método	Propósito
RescueCaseRepository	findByCaseCode(String caseCode)	Buscar un caso por su código
RescueCaseRepository	findByRescueCenterCode(String code)	Casos pertenecientes a un centro
RescueCaseRepository	findByStatusOrderByRescueDateAsc(RescueStatus status)	Casos según estado, ordenados por fecha

AnimalRepository	findByAnimalCode(String animalCode)	Buscar un animal por su código
AnimalRepository	findByRescueCaseRescueCenterCode(String code)	Animales de un centro determinado (navegación de propiedades)
AnimalRepository	findByCommonNameContainingIgnoreCase(String name)	Animales cuyo nombre común contiene un texto, sin distinguir mayúsculas
SpecialistRepository	<i>(ver @Query abajo — se usa JPQL para expertise por complejidad de la relación N:M)</i>	
TreatmentRepository	findByAnimalIdOrderByPerformedAtAsc(Long animalId)	Tratamientos de un animal, ordenados cronológicamente
TreatmentRepository	findByPerformedAtBetween(LocalDateTime start, LocalDateTime end)	Tratamientos realizados en un intervalo de fechas

9. Consultas JPQL implementadas

SpecialistRepository.findActiveByExpertise

@Query("""

select s

from Specialist s

```

join s.expertiseAreas e
where s.active = true
and lower(e.name) = lower(:expertiseName)
""")

```

```

List<Specialist> findActiveByExpertise(@Param("expertiseName") String
expertiseName);

```

Especialistas activos que poseen una determinada área de experiencia.

TreatmentRepository.findBySpecialistExpertise

```

@Query("""
select t
from Treatment t
join t.specialist s
join s.expertiseAreas e
where lower(e.name) = lower(:expertiseName)
""")

```

```

List<Treatment> findBySpecialistExpertise(@Param("expertiseName") String
expertiseName);

```

Tratamientos realizados por especialistas con una determinada experiencia.

AnimalRepository.findInRehabilitationTreatedBySpecialistWithExpertise (reto integrador)

```

@Query("""
select distinct a
from Animal a
join a.rescueCase rc
join a.treatments t
join t.specialist s
join s.expertiseAreas e

```

```

where rc.status = :status

and lower(e.name) = lower(:expertiseName)

""")

List<Animal> findInRehabilitationTreatedBySpecialistWithExpertise(

    @Param("status") RescueStatus status,

    @Param("expertiseName") String expertiseName

);

```

Animales en un determinado estado de rescate que hayan recibido al menos un tratamiento de un especialista con cierta experiencia. Se optó por @Query + JPQL en lugar de un Query Method derivado porque la consulta atraviesa cuatro relaciones encadenadas (incluyendo dos colecciones), lo que haría el nombre del método ilegible y no permitiría controlar explícitamente el DISTINCT necesario para evitar animales duplicados.

Estructura del proyecto

deepblue-rescue/

```

|
|
|— pom.xml
|
|— src/main/java/com/deepblue/rescue
|   |— DeepblueRescueApplication.java
|   |— domain/    (RescueCenter, RescueCase, Animal, MedicalRecord, Specialist,
Expertise, Treatment y sus enums)
|   |— repository/ (interfaces JpaRepository con Query Methods y @Query)
|   |
|   |— src/main/resources
|       |— application.yml
|       |— db/migration/ (V1, V2, V3)

```

```
|
|└─ src/test/java/com/deepblue/rescue
|   └─ PersistenceIntegrationTest.java
|
|└─ README.md
```